

מבוא למדעי המחשב

שיעור 10: מדעי המחשב, חישוביות וסיבוכיות

לעיון נוסף:

- משחק החיקוי (The Imitation Game). במאי: Morten Tyldum. 2014.
- <https://www.yahoo.com/tech/our-interview-with-turing-test-winner-eugene-goostman-88482732919.html>
- Charles Babbage. *Table of the Logarithms of the Natural Numbers from 1 to 108000*. William Clowes and Sons, 1841.
- P.M. Hill and J.W. Lloyd. *The Gödel Programming Language*. MIT Press, 1994.
- <http://compsoc.dur.ac.uk/whitespace/>
- <http://www.nyx.net/~gthompso/quine.htm>
- <http://www.claymath.org/millennium-problems/p-vs-np-problem>

קצת על הבחינה

מה אתם צריכים לדעת

- אנחנו נניח שאתם מכירים את כל המבנים הבסיסיים של פייתון שלימדנו. זה כולל:
 - מושגים, כגון immutable, איטרטור, תחום הגדרה של משתנה, מצביע וכיוב'.
 - אופרטורים, כגון +, *, %, <, and, in וכיוב'.
 - פקודות, כגון if, while, break, return, class, def, yield וכיוב'.
 - מחלקות מהספריה של פייתון, כגון list, tuple, str, set, dict וכיוב'.
 - פקודות פייתון שמבצעות פעולות עם אובייקטים מהמחלקות הללו, כגון אתחול, שרשור, גישה לאבר, slicing, list comprehension וכיוב'.
 - פונקציות ספריה סטנדרטיות (לא דברים אזוטריים)
- כלים שלימדנו לפתרון בעיות, כגון רקורסיה, מיון, רשימה משורשרת וכיוב'.
- ניתוח ביצועים (סיבוכיות):
 - הבנת הסימונים האסימפטוטיים.
 - הבנת ניתוח הסיבוכיות של האלגוריתמים שניתחנו.
 - כתיבת קוד על פי דרישות סיבוכיות וניתוח קוד כתוב (במסגרת הטיעונים המתמטיים שראיתם בשיעורים ובתרגולים).

דרישות מהפטרון

- לא נוריד נקודות על שגיאות תחביר בקוד שאינן מעידות על חוסר הבנה.
- אין צורך לכתוב תיעוד כפי שנדרשתם בתרגילים. אפשר להוסיף הסברים בעברית אם נראה לכם נחוץ להבנת הפטרון.
- השתדלו לכתוב קוד קצר, קריא ויעיל (וכמובן עונה לשאלה שנשאלתם).
- אין צורך להוסיף לפטרון (הוכחות, נוסחאות, דוגמאות, ...) מעבר למה שנדרשתם מפורשות בשאלה.

הכנה וגישה לבחינה

- כדאי לעבור על החומר של השיעורים והתרגולים.
- נסו לפתור לפחות חלק מהתרגילים שקיבלתם מחדש, תוך שימוש בכלים מתקדמים יותר שלמדתם מאז.
- נסו לפתור את הבחינות הקודמות. שימו לב שיש נושאים שלא נלמדו השנה (ולחיפך).
- בבחינה עצמה כדאי לחפש ולענות קודם על השאלות שנראות לכם קלות יותר, אבל להקפיד להשאיר די זמן לשאלות הקשות יותר.

בהצלחה!

מדעי המחשב

מדעי המחשב

תיאוריה

קורסים: מבני נתונים, אלגוריתמים, חישוביות, קריפטוגרפיה

מערכות מחשב

קורסים: מ-NAND ל-Tetris, מערכות הפעלה, רשתות תקשורת, מסדי נתונים, גרפיקה

הנדסת תוכנה

קורסים: תכנות מונחה עצמים, סדנת תכנות C/C++, הנדסת תוכנה

בינה מלאכותית

קורסים: מערכות לומדות, בינה מלאכותית, עיבוד תמונות

חישוב ומדע

קורסים: חישוב קוונטי, כלכלה וחישוב, ביולוגיה חישובית, חישוב נומרי

חישוב כתופעת טבע



קצת הסטוריה ...



Gottfried Wilhelm Leibniz 1646-1718

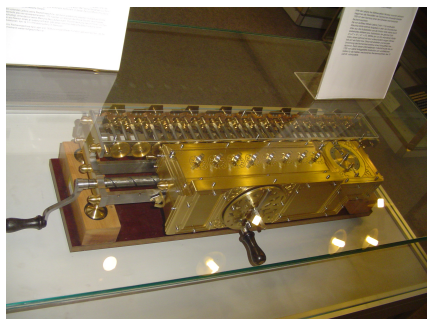
חזון: מיכון החשיבה הלוגית

שני מרכיבים:

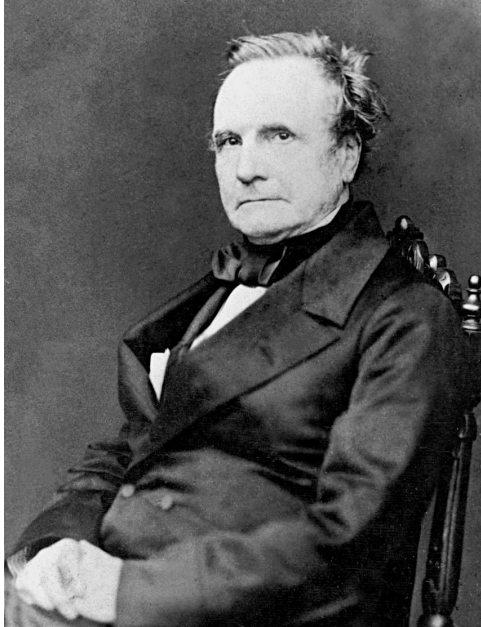
- שפה פורמלית אוניברסלית *Characteristica Universalis*
- תחשיב לוגי *Calculus Ratiocinator*

מכונת החישוב (*Staffelwalze*) של לייבניץ

פעולות חשבון: חיבור, חיסור, כפל, חילוק



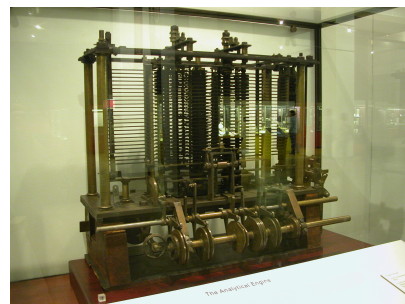
חישוב מכני



Charles Babbage 1791-1871



1822 מנוע ההפרשים
חישובים נומריים: טורי טיילור



1834 המנוע האנליטי
חישוב כללי

הגדרה מתמטית של חישוב

1920 התוכנית של הילברט David Hilbert

להוכיח שהמתמטיקה היא עקבית, שלמה וכריעה

1931 הקידוד של גדל Kurt Gödel

משפטי אי-השלמות

אם תורת המספרים עקבית, אז יש משפט שלא ניתן להוכיחו או להפריכו
(נדרשת "שיטה אפקטיבית" לבדוק הוכחות - מערכת לוגית אפקטיבית)

1936 מכונת טיורינג Alan Turing

קיימת מכונה אוניברסלית

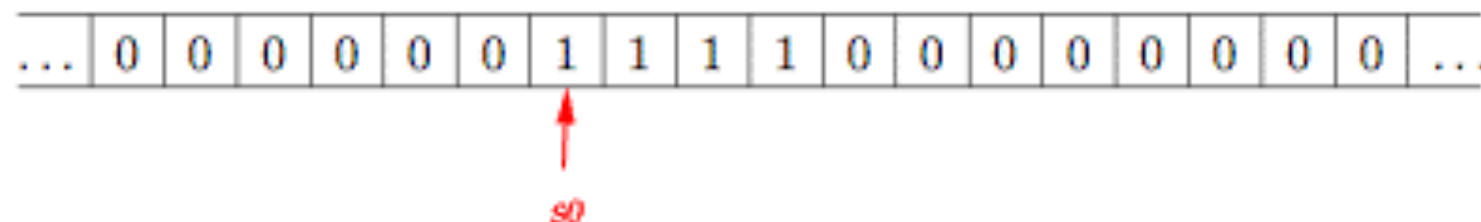
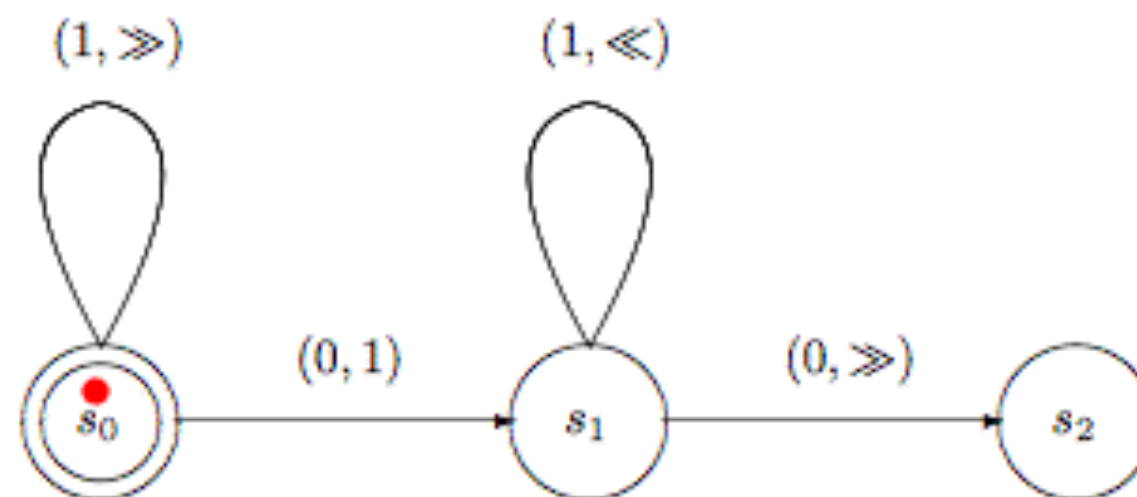
אי אפשר להכריע את בעיית העצירה (שניתנת לניסוח בתורת המספרים)

תחשיב λ Alonzo Church

מכונת טיורינג

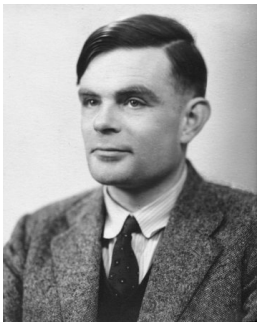


Alan Mathison Turing 1912-1954



עוד על מכונת טיורינג

- לכל אלגוריתם צריך לפתח אוטומט בקרה משלו.
- טיורינג הוכיח שיש מכונה אוניברסלית שהקלט שלה זה זוג (machine, input) והיא מפעילה את machine על input.
- לצורך זה נדרש קידוד סטנדרטי של מכונות.
- האינטרפרטר של פייתון הוא דוגמה למכונה אוניברסלית. אפשר להריץ דרכו כל תוכנית בפייתון על כל קלט.



התזה של צ'רץ' וטיורינג



- כל מה שניתן לחישוב, ניתן לחישוב על ידי מכונת טיורינג.
- פרשנות רעיונית: ההגדרה הפורמלית של מכונת טיורינג מזדהה עם המושג האינטואיטיבי של חישוב/אלגוריתם.
- פרשנות פיסיקלית: לא ניתן לבנות מכשיר שמבצע חישוב שאינו אפשרי במכונת טיורינג.
- פרשנות מעשית: תוכנית מחשב בפייתון שקולה למכונת טיורינג.
- גרסה מודרנית של התזה: אין מנגנון חישוב שמשמעותית יעיל יותר בזמן או מקום.
- האם יש אלגוריתם יעיל שמוצא הוכחות מתמטיות קצרות למשפטים שיש להם הוכחה כזו?

שאלת הדיוק

מודל העולם הפיסיקלי הוא
אנלוגי: התהליכים והערכים
הם רציפים
בפרט צצים ערכים בדיוק **אינסופי**

חישוב הוא **דיגיטלי**:
מתבצע בצעדים בדידים
על נתונים בדידים
שברים מיוצגים בדיוק **סופי**

- איך מצדיקים את האוניברסליות של חישוב דיגיטלי? -
 - אי אפשר למדוד אות פיסיקלי בדיוק אינסופי
 - במערכות דינאמיות "נחמדות" השגיאה כתוצאה מדיוק סופי לא מתנפחת מהר.
- אבל יש מערכות דינאמיות כאוטיות

תורת הקוונטים

- האם מכניקת הקוונטים ניתנת לדימוי על מכונת טיורינג?
- חישובית - כן, מבחינת היעילות - לא ידוע.
- ידוע אלגוריתם קוונטי יעיל לפירוק מספר לגורמים ראשוניים. לא ידוע אלגוריתם כזה במודל של מכונת טיורינג.
- האם אפשר לבנות מחשב קוונטי?
- זה האתגר המשמעותי ביותר לתזה של צ'רץ'-טיורינג.

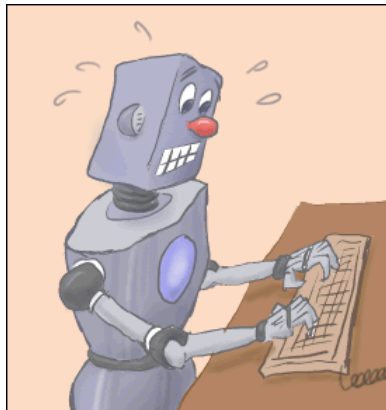
המוח

- האם המוח הוא רק מחשב? -

- בפילוסופיה - בעיית גוף-נפש:

- מוניזם (מטריאליזם): יש רק חומר.

- דואליזם: יש עוד משהו (נפש, תודעה, הכרה, ...)



- מבחן טיורינג:

- אין עדיין תוכנית מחשב שהצליחה במבחן.

- התוכנה הרוסית Eugene Goostman הצליחה ב-2014 לשכנע שלישי

מהשופטים במבחן טיורינג שהיא ילד אוקראיני בן 13.

- יש מטלות שכיום מחשב עושה טוב מאדם: חישובים אריתמטיים, משחק

שח, המרת דיבור לטקסט, משחק גו, זיהוי פרצופים (?), ...

המוח מול הענן של גוגל

גוגל

מוח

10^7 עד 10^9 ליבות

10^9 עד 10^{10} נוירונים

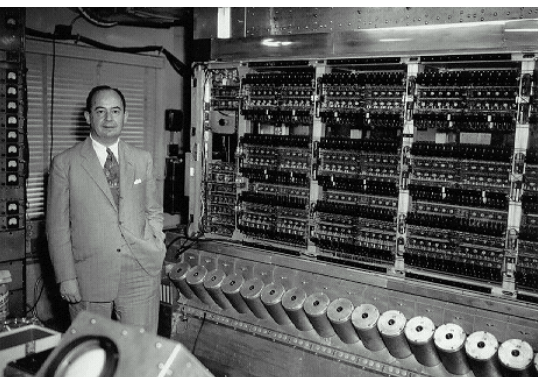
כל נוירון מחובר ל-1000 עד 10,000
נוירונים אחרים

קצב חישוב של ליבה:
 10^8 עד 10^{10} פעולות בשנייה

קצב "ירייה" של נוירון:
 10^0 עד 10^2 פעמים בשנייה

סה"כ 10^{15} עד 10^{19} פעולות בשנייה

סה"כ 10^{12} עד 10^{16} פעולות בשנייה



מחשב אוניברסלי

George Boole

1848 אלגברה בוליאנית

Claude Shannon

1937 מעגלים דיגיטליים

ניתן לממש אלגברה בוליאנית ע"י מערכים של ממסרי טלפוניה אלקטרו-מכניים

John von Neumann

1945 EDVAC

הארכיטקטורה של פון-נוימן (stored-program computer):
התוכנית מאוכסנת בזיכרון כמו הנתונים.

Claude Shannon

1948 תורת האינפורמציה

קודים בינאריים לתיקון שגיאות.

חִשְׁבוּבִיּוֹת

בעיית העצירה

בעיית העצירה

- לולאה אינסופית - הפונקציה לא עוצרת:

```
def ad_infinitem():  
    while (True):  
        print("again ???!!!")
```

בעיית העצירה

- לולאה אינסופית - הפונקציה לא עוצרת:

```
def ad_infinity():  
    while (True):  
        print("again ???!!!")
```

- פונקציה שכן עוצרת:

```
def mortal():  
    while (False):  
        print("Never once!")
```

בעיית העצירה

- אפשר להבדיל בין שני המקרים?

- שאלה מבחינה של פרופסור יובל המבולבל בקורס מבוא למדעי המחשב:

- השלימו את הקוד של הפונקציה הבאה:

```
def halt(f, data):
```

```
...
```

- הקלט ל-halt הוא פונקציה f בפייתון ו- $data$ שהוא קלט עבור f (אם f לא מקבלת קלט, נעביר $data = None$).

- הפונקציה $halt$ מחזירה $True$ אם הקריאה $f(data)$ עוצרת ו- $False$ אחרת.

בעיית העצירה

- בפרט, הקריאה

halt(ad_infinity, None)

צריכה להחזיר False.

- הקריאה

halt(mortalis, None)

צריכה להחזיר True.

כריעות

- נתבונן בקוד הבא:

```
def paradox(f):  
    # f is a Python function  
    if halt(f, f):  
        while (True):  
            pass  
    else:  
        return
```

- הקריאה `paradox(f)` עוצרת אם ורק אם `f(f)` לא עוצרת.

כריעות

- מה יקרה אם נריץ את הקוד הבא? - `paradox(paradox)`
- אם `halt(paradox, paradox)` מחזירה `True`, אז `paradox(paradox)` לא עוצרת.
- אם `halt(paradox, paradox)` מחזירה `False`, אז `paradox(paradox)` עוצרת.
- בשני המקרים נקבל סתירה!
- מסקנה: אי אפשר לכתוב את `halt` !!!

מסקנה

- יש בעייה חישובית מוגדרת היטב שאין אף אלגוריתם שפותר אותה!
- יש עוד בעיות כאלה.
- האפשרות לשכפל את הקלט ולתת לפונקציה את עצמה כקלט (הפנייה עצמית) היא קריטית לטיעון.

קוד שמדפיס את עצמו (quine)

Python 3.4.1: Quine.py - /Users/yrabani/Documents/14 פייתון/תשעה/שיעור 14/Code/Quine.py

```
a = ['print("a =", a)', 'for s in a:\n    print(s)']
print("a =", a)
for s in a:
    print(s)
```

Ln: 1 Col: 42

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
a = ['print("a =", a)', 'for s in a:\n    print(s)']
print("a =", a)
for s in a:
    print(s)
>>>
```

Ln: 10 Col: 4

(Omar Antolin omar@galois.fciencias.unam.mx מבוסס על קוד של)



משוואות דיופנטיות



- השאלה העשירית של הילברט: למצוא אלגוריתם שמזהה אם למערכת משוואות דיופנטיות יש פיתרון.

$$x^3 y^2 z + xwz - 3 = 0$$

$$x^2 z + 3w^2 z^3 - 2z = 0$$

...

$$z^7 w - zw^6 - 2 = 0$$

- בהינתן תוכנית מחשב, יש אלגוריתם שממיר אותה למערכת משוואות דיופנטיות שיש לה פיתרון אםם התוכנית עוצרת.

- מסקנה: אין פיתרון לבעייה העשירית של הילברט!

משפט אי השלמות של גדל

- מערכת הוכחה יעילה מאפשרת לכתוב את הפונקציה הבאה (הקלט הוא שתי מחרוזות):

```
def is_proof(theorem, proof):
```

- נזדקק גם לאיטרטור אינסופי שעובר על כל המחרוזות (לפי אורך ובכל אורך בסדר לקסיקוגרפי):

```
class AllStrings:
```

- לבסוף נזדקק לפונקציות שמייצרות טיעונים פורמליים (כמחרוזות):

```
def halt_thm(f, data):
```

```
def nohalt_thm(f, data):
```

מימוש AllStrings

Python 3.4.1: AllStrings.py - /Users/yrabani/Documents/14 פייתון/תשעה/שיעור - המחשב - Code/AllStrings.py

```
class AllStrings:
    def __init__(self):
        pass
    def __iter__(self):
        def allstrings_generator():
            number_of_bits = 1
            while True:
                for number in range(2**number_of_bits):
                    yield padded_binary_representation(number, number_of_bits)
                number_of_bits += 1

            return allstrings_generator()

def padded_binary_representation(nonnegative_int, number_of_bits):
    output = ''
    if nonnegative_int < 2**number_of_bits:
        for bit in range(number_of_bits):
            output = str(nonnegative_int%2) + output
            nonnegative_int //= 2

    return output
```

Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)

[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin

Type "copyright", "credits" or "license()" for more information.

>>> ===== RESTART =====

>>>

>>> iterator = iter(AllStrings())

>>> next(iterator)

'0'

>>> next(iterator)

'1'

>>> next(iterator)

'00'

>>> next(iterator)

'01'

>>> next(iterator)

'10'

>>> next(iterator)

'11'

>>> next(iterator)

'000'

>>> next(iterator)

'001'

>>> next(iterator)

'010'

>>> next(iterator)

'011'

>>> next(iterator)

'100'

>>>

המשך

בהנחה שמערכת ההוכחה שלנו שלמה:

```
def halt(f, data):  
    halt_statement = halt_thm(f, data)  
    nohalt_statement = nohalt_thm(f, data)  
    for proof in AllStrings():  
        if is_proof(halt_statement, proof):  
            return True  
        elif is_proof(nohalt_statement, proof):  
            return False
```

סיבוכיות

תזכורת

- סיבוכיות זמן: לכל אורך קלט - כמה פעולות נדרשות לחישוב במקרה הגרוע ביותר.
- סיבוכיות מקום: לכל אורך קלט - כמה תאי זיכרון (חוץ מהקלט) נדרשים.
- במכונת טיורינג, פעולה בסיסית זה צעד אחד של המכונה. תא זיכרון הוא משבצת אחת בסרט האינסופי.
- בקורס אנחנו משתמשים במודל מעורפל יותר - כל פעולת מחשב שלוקחת זמן שלא תלוי בגודל הקלט, כל משתנה בודד שגודלו קבוע ולא תלוי בגודל הקלט.

גרסה כמותית

- השלימו את הקוד של הפונקציה הבאה:

```
def halt_in(steps, f, len):
```

```
...
```

- הקלט ל-halt_in הוא מספר טבעי steps, פונקציה f בפייתון, ומספר טבעי len.
- הפונקציה halt_in מחזירה True אם קיים קלט data באורך len עבורו הקריאה f(data) עוצרת אחרי לכל היותר steps פעולות בסיסיות, או False אחרת.

המחלקה NP

- בוודאי יש אלגוריתם לפתור את הבעיה: נעבור בסדר לקסיקוגרפי על כל הקלטים באורך len ונבדוק כל אחד.
- זה לא אלגוריתם יעיל: יש 2^{len} קלטים אפשריים (בהנחה שמדובר ברצפים של ביטים).
- האם יש אלגוריתם שהסיבוכיות שלו תלוייה רק פולינומית ב- len (וב-steps)?

Stephen Cook

1971 זה יפתור המון בעיות

בפרט, את כל הבעיות ב-NP.

NP - הבעיות עבורן לכל קלט שמחזיר True יש עד לזה שקל לבדוק.

רדוקציות

- הפונקציה הבאה מקבלת בדיוק אותו קלט כמו `halt_in`, והיא מתרגמת אותו לקלט לבעייה אחרת X (שיש לה פלט True או False):

`def compile_to_X(steps, f, len):`

- הפונקציה `compile_to_X` מבטיחה את התכונה הבאה: אם פונקציה $x(data)$ פותרת את הבעייה X על כל קלט `data`. אז לכל קלט `steps, f, len` מתקיים:
 $x(compile_to_X(steps, f, len)) == halt_in(steps, f, len)$

1972 מימוש יעיל עבור 21 בעיות Richard Karp

הבעיות הללו קשות לחישוב לפחות כמו הבעייה של Cook.
מאז נתגלו אלפי בעיות כאלה.